

The Authorization Policy Existence Problem

Pierre Bergé¹, Jason Crampton, Gregory Gutin², and Rémi Watrigant

Abstract—Constraints such as separation-of-duty are widely used to specify requirements that supplement basic authorization policies. However, the existence of constraints (and authorization policies) may mean that a user is unable to fulfill her/his organizational duties because access to resources has been denied. In short, there is a tension between the need to protect resources (using policies and constraints) and the availability of resources. Recent work on workflow satisfiability and resiliency in access control asks whether this tension compromises the ability of an organization to achieve its objectives. In this paper, we develop a new method of specifying constraints which subsumes much related work and allows a wider range of constraints to be specified. The use of such constraints leads naturally to a range of questions related to “policy existence”, where a positive answer means that an organization’s objectives can be realized. We analyze the complexity of these policy existence questions and, for particular sub-classes of constraints defined by our language, develop fixed-parameter tractable algorithms to solve them.¹

Index Terms—Access control, resiliency, satisfiability, computational complexity, fixed-parameter tractability

1 INTRODUCTION

ACCESS control is a fundamental aspect of the security of any multi-user computing system, and is typically based on the specification and enforcement of an authorization policy. Such a policy identifies which interactions between users and resources are to be allowed by the system.

Over the last twenty years, authorization policies have become more complex, not least because of the introduction of constraints, which further refine an authorization policy. A separation-of-duty constraint (also known as the “two-man rule” or “four-eyes policy”) may, for example, require that no single user is authorized for some particularly sensitive group of resources. Such a constraint is typically used to prevent misuse of the system by a single user.

The use of authorization policies and constraints, by design, limits which users may access resources. Nevertheless, the ability to perform one’s duties requires access to particular resources, and overly prescriptive policies and constraints may mean that some resources are inaccessible. In short, “tension” may exist between authorization policies and operational demands: too lax a policy may suit organizational

demands but lead to security violations; whereas too restrictive a policy may compromise an organization’s ability to meet its business objectives.

Recent work on workflow satisfiability and access control resiliency has recognized the importance of being able to identify whether or not security policies prevent an organization from achieving its objectives [2], [3], [4], [5], [6]. In this paper, we seek to extend existing work in this area. Specifically, we introduce the AUTHORIZATION POLICY EXISTENCE PROBLEM (APEP), which may be treated as a decision or optimization problem. Informally, APEP seeks to find an authorization policy, subject to restrictions on individual authorizations (defined by a “base” authorization relation) and restrictions on collective authorizations (defined by a set of authorization constraints). We show that a number of problems in the literature on workflow satisfiability and resiliency are special cases of APEP, thereby showing that APEP is computationally hard.

The framework within which APEP is defined admits a greater variety of constraints than is usually considered in either the standard access control literature [7], [8], [9], [10] or in workflow satisfiability [6], [11], [12]. In this paper we characterize the constraints of interest and extend the definition of user-independent constraints [12] to this framework. We then establish the complexity of APEP for certain types of constraints, using both classical and multi-variate complexity analysis. In this paper, we make some progress in this direction by establishing the complexity of APEP for the constraints that we believe will be the most useful in practice. In particular, we establish connections between APEP and both the WORKFLOW SATISFIABILITY PROBLEM and resiliency in access control.

In the next section, we summarize relevant background material and related work. We introduce the APEP in Section 3 and elaborate on the nature of the constraints we consider in Section 4. We then discuss further constraint types and connections between APEP and existing problems in the literature. In Section 6, we investigate the complexity of several

1. An extended abstract of this paper appeared in the Proceedings of the Seventh ACM Conference on Data and Application Security and Privacy [1]. Research was partially supported by Leverhulme Trust grant RPG-2018-161 and Royal Society Wolfson Research Merit Award.

- P. Bergé is with LRI, Université Paris-Sud, Université Paris-Saclay, Saint-Aubin 91190, France. E-mail: pierre.berge@lri.fr.
- J. Crampton is with the Department of Information Security, Royal Holloway, University of London, London WC1E 7HU, United Kingdom. E-mail: jason.crampton@rhul.ac.uk.
- G. Gutin is with the Department of Computer Science, Royal Holloway, University of London, London WC1E 7HU, United Kingdom. E-mail: G.Gutin@rhul.ac.uk.
- R. Watrigant is with Université de Lyon, UCBL1, CNRS, LIP, ENS de Lyon, 69100 Villeurbanne, France. E-mail: remi.watrigant@univ-lyon1.fr.

Manuscript received 13 Oct. 2017; revised 24 Oct. 2018; accepted 15 Nov. 2018. Date of publication 28 Nov. 2018; date of current version 12 Nov. 2020. (Corresponding author: Gregory Gutin.)
Digital Object Identifier no. 10.1109/TDSC.2018.2883416

variants of the APEP. We conclude the paper with a summary of our contributions and some ideas for future work.

2 BACKGROUND AND RELATED WORK

A number of interesting (and computationally hard) problems arise naturally in the context of authorization policies and constraints. However, the relative sizes of the parameters in many of these problems mean that it is fruitful to analyze these problems using multivariate complexity analysis. In this section, we review some of those problems and provide a brief introduction to fixed-parameter tractability.

2.1 Fixed-Parameter Tractability

Many problems take multiple inputs and the complexity of solving such problems is determined by the sizes of those inputs. In general, a problem may be hard in terms of the total size of the input. However, if we consider the complexity of a problem under an assumption that some of the parameters of the input are small and terms that are exponential in those parameters are acceptable, then we may discover that relatively efficient algorithms exist to solve the problem.

More formally, an algorithm is said to be *fixed-parameter tractable* (FPT) if it solves a decision problem in time $O(f(k)p(n))$, where k is some (small) parameter of the problem, n is the total size of the input, and f and p are, respectively, a computable function and a polynomial. As is customary in the literature on FPT algorithms, we will often write $O(f(k)p(n))$ as $O^*(f(k))$. (That is, O^* suppresses polynomial factors, as well as multiplicative constants.) If a problem can be solved using an FPT algorithm then we say that it is an *FPT problem* and that it belongs to the class FPT [13], [14]. An FPT algorithm for a hard problem is particularly valuable when k is significantly smaller than n for most instances of the problem that arise in practice.

While many NP-complete problems admit FPT algorithms, many other NP-complete problems do not admit such algorithms unless certain parameterized complexity-theory assumptions fail, which is highly unlikely [13], [14]. In particular, the WORKFLOW SATISFIABILITY PROBLEM is highly unlikely to admit an FPT algorithm [6]. However, if all constraints are user-independent, then the problem is FPT and efficient algorithms have been developed to solve practical instances of the problem [12], [15], [16]. Since the APEP is an extension of the WORKFLOW SATISFIABILITY PROBLEM, it is also highly unlikely to admit an FPT algorithm. In this paper we identify important restrictions to the general APEP which do admit such algorithms.

2.2 Workflow Satisfiability

A workflow may be modeled as a set of steps in some automated business process. An authorization relation determines which users are authorized to perform which steps, and constraints restrict which subsets of users may perform subsets of steps [6], [17], [18]. Given a set of users U , a set of workflow steps S , an authorization relation $A \subseteq S \times U$, and a set of constraints C , a plan $\pi : S \rightarrow U$ is a function allocating users to steps. A valid plan must allocate an authorized user to each step and every constraint must be satisfied. The WORKFLOW SATISFIABILITY PROBLEM

(WSP) asks whether there exists a valid plan for a given U , S , A and C . Basin, Burri and Karjoth model a workflow (with “break points”) as a process algebra and introduced the notion of an *enforcement process*, which is analogous to a valid plan [11]. This leads naturally to the *enforcement process existence* problem, which is analogous to the workflow satisfiability problem and inspires the name for the problem we study in this paper.

Wang and Li observed that fixed-parameter algorithmics is an appropriate way to study WSP, because the number of steps is usually small and often much smaller than the number of users.² Wang and Li [6] proved that WSP is FPT if we consider only separation-of-duty and binding-of-duty constraints [6]. We will write such constraints as $(s, s', \neq)$ and $(s, s', =)$, respectively, where $s, s' \in S$. A plan π satisfies a constraint $(s, s', \neq)$ ($(s, s', =)$, respectively) if $\pi(s) \neq \pi(s')$ ($\pi(s) = \pi(s')$, respectively). WSP with only separation-of-duty constraints (only binding-of-duty constraints, respectively) will be denoted by WSP($\neq$) (WSP($=$), respectively). Subsequent research has shown that WSP remains FPT even if additional types of constraints, notably user-independent [12] and class-independent constraints [20], are permitted in the input to WSP. Note that WSP is not FPT in general [6] unless a widely-accepted hypothesis in complexity theory fails; a significant body of research suggests this is highly unlikely [13].

2.3 Static Separation-of-Duty Constraints

Constraints have been studied extensively in the context of role-based access control (RBAC) [8], [9], [10], [21]. In its simplest form, a static separation-of-duty constraint may be defined as a pair of roles $\{r, r'\}$ belonging to the set of roles R . A user-role assignment relation $UR \subseteq U \times R$, where U is the set of users, satisfies the constraint $\{r, r'\}$ if there is no user u such that (u, r) and (u, r') belong to UR .

More generally, Li, Tripunitara and Bizri [9] introduced the notion of a k -out-of- n static separation-of-duty constraint, defined as a pair (R', n) , where R' is a subset of R of cardinality k . A user-role assignment relation $UR \subseteq U \times R$ satisfies the constraint (R', n) if there is no set of t users, where $t < k$, that are collectively authorized for R' . That is, for all subsets V of U such that $|V| < k$,

$$\bigcup_{u \in V} \{r : (u, r) \in UR\} \subset R'.$$

Note that the simple separation-of-duty constraint defined by a pair of roles $\{r, r'\}$ is a 2-out-of-2 separation-of-duty constraint.

2.4 Resiliency

Li, Wang and Tripunitara introduced the idea of resiliency in access control [4]. Informally, a resiliency policy is a requirement that even in the absence of a limited number of users the remaining users can be authorized for some set of resources such that given constraints are satisfied. The existence of both a resiliency policy and authorization

2. The SMV loan origination workflow studied by Schaad et al., for example, has 13 steps and identifies five roles [19]. It is generally assumed that the number of users is significantly greater than the number of roles.

constraints may mean that no authorization relation can satisfy all requirements. Li et al. introduce a number of problems investigating whether an authorization relation does exist for a given resiliency policy [4].

3 THE AUTHORIZATION POLICY EXISTENCE PROBLEM

In this paper, we extend existing work on workflow satisfiability, constraints and resiliency, by defining a simple yet very expressive authorization framework. Roughly speaking, we specify a problem concerned with the existence of an appropriate authorization relation.

Given a set of users U and a set of resources R to which access should be restricted, we may define an *authorization relation* $A \subseteq U \times R$, where $(u, r) \in A$ if and only if u is authorized to access r . Given a resource r , we will write $A(r)$ to denote the set of users that are authorized to access resource r . More formally, $A(r) = \{u \in U : (u, r) \in A\}$. Similarly, for $u \in U$, we will write $A(u)$ to denote the set of resources that u is authorized to access, that is $A(u) = \{r \in R : (u, r) \in A\}$. We extend this notation to subsets of R and U in the natural way: for $R' \subseteq R$ and $U' \subseteq U$,

$$A(R') \stackrel{\text{def}}{=} \bigcup_{r \in R'} A(r) \quad \text{and} \quad A(U') \stackrel{\text{def}}{=} \bigcup_{u \in U'} A(u).$$

We introduce two fundamental concepts, which will be used to formulate the AUTHORIZATION POLICY EXISTENCE PROBLEM.

- a *base authorization relation* $A_{\text{Bse}} \subseteq U \times R$ such that $A_{\text{Bse}}(r) \neq \emptyset$ for each $r \in R$;
- a set of *authorization constraints* C .

Informally, A_{Bse} specifies restrictions on all valid authorization relations, while C specifies additional restrictions that any valid authorization relation must satisfy.

An authorization constraint may be defined by enumerating the set of all authorization relations that satisfy the constraint. Of course, an extensional definition of this nature is utterly impractical, and all useful constraints will be defined in an intensional manner. A simple example would be a constraint requiring no user is assigned to both resources r and r' . In other words, an authorization relation A satisfies this constraint provided that $\{(u, r), (u, r')\} \not\subseteq A$ for all $u \in U$. We discuss constraints in more detail in Section 4.

More formally, we have the following definitions. Given a base authorization relation A_{Bse} and a set of constraints C , we say an authorization relation $A \subseteq U \times R$ is

- *authorized* with respect to A_{Bse} if $A \subseteq A_{\text{Bse}}$;
- *complete* if $A(r) \neq \emptyset$ for every $r \in R$;
- *eligible* with respect to $c \in C$ if A satisfies c ;
- *eligible* with respect to C if A satisfies c for all $c \in C$;
- *valid* with respect to A_{Bse} and $c \in C$ if A is authorized, complete and eligible with respect to c ; and
- *valid* with respect to A_{Bse} and C if A is authorized, complete and eligible with respect to C .

We introduce the term AUTHORIZATION POLICY EXISTENCE PROBLEM as a generic term for questions concerned with finding a valid authorization relation, given A_{Bse} and C . Then APEP comes in two “flavors”: (a) *Decision* (D-APEP):

Does there exist a valid authorization relation? If so, find a valid authorization relation. (b) *Optimization* (O-APEP): Find a “best” valid authorization relation if one exists (where the objective function has to be specified).

We assume that determining whether an authorization relation satisfies a constraint takes polynomial time. (This is a reasonable assumption for all constraints of relevance to access control.) Let n denote $|U|$, k denote $|R|$ and m denote $|C|$. Then a brute force approach to solving D-APEP (by simply examining every possible authorization relation) takes time $O^*(2^{nk})$.

A few special cases of APEP are worth mentioning. For simplicity we will write A_{Sol} to denote one of the authorization relations that are solutions to APEP.

- 1) A_{Sol} is required to be a *function* $A_{\text{Sol}} : R \rightarrow U$. In this case, we allocate a unique user to each resource. Computing a plan allocating one user to each step in a workflow instance, subject to an authorization policy defined by A_{Bse} and some constraints C , is an example of this type of scenario. In this case, D-APEP corresponds to the WORKFLOW SATISFIABILITY PROBLEM (WSP) [6]. Moreover, the CARDINALITY-CONSTRAINED MINIMUM USER PROBLEM [22], whose solution is a plan using the minimum number of users, is an instance of O-APEP.
- 2) $A_{\text{Bse}} = U \times R$. In this case, A_{Bse} itself imposes no restrictions on A_{Sol} . All restrictions on A_{Sol} are defined by the constraints C . Defining an authorization policy in the presence of static separation-of-duty constraints is an example of this type of scenario. Li, Tripunitara and Bizri have studied problems of this nature [9].
- 3) A constraint that requires each resource is assigned to at least t users enables us to model problems related to resiliency in access control [4] and workflow systems [6].
- 4) If we seek to maximize the cardinality of A_{Sol} , then, informally, a solution to O-APEP provides a “resilient” authorization policy. While this is different from existing notions of resiliency [4], [6], it would seem to be an interesting way of approaching the problem of making an authorization policy resilient to the unavailability of users.

4 CONSTRAINTS

We now describe constraints in more detail. We generalize the approach used in prior work on constraints for workflow systems [3], [6].

4.1 Binding-of-Duty and Separation-of-Duty Constraints

Binding-of-duty and *separation-of-duty* constraints have received considerable attention in the access control literature, and such constraints may be *static* or *dynamic*. Informally, static constraints specify restrictions on policy relations, whereas dynamic constraints specify constraints on particular sequences of events within the context of an access control system. A (static) separation-of-duty constraint, for example, in the context of role-based access control system, might

require that no user is authorized for both roles r and r' [10]. In contrast, a (dynamic) separation-of-duty constraint, in the context of a workflow system, might simply require that two steps s and s' are performed by different users in each instance of the workflow [6], [18]. (This constraint, however, does not prevent the same user being authorized for both those steps.) Within the framework of APEP, we seek to define a more general (and uniform) syntax and semantics for constraints.

We express constraints in terms of the following logical (binary) operators defined via their respective truth tables:

p	q	$p \leftrightarrow q$	$p \rightarrow q$	$p \leftarrow q$	$p \updownarrow q$
0	0	1	1	1	0
0	1	0	1	0	1
1	0	0	0	1	1
1	1	1	1	1	0

Let r and r' be resources in R ; let $\circ$ denote one of the logical operators in the set $\{\leftrightarrow, \leftarrow, \rightarrow, \updownarrow\}$; and let Q be one of the first-order quantifiers $\exists$ or $\forall$. Then $(r, r', \circ, Q)$ is a constraint: a constraint of the form $(r, r', \circ, \forall)$ is said to be *universal*, while a constraint of the form $(r, r', \circ, \exists)$ is said to be *existential*. A complete relation A

- satisfies $(r, r', \circ, \exists)$ if there exists $u \in A(r) \cup A(r')$ such that the propositional formula $(u \in A(r)) \circ (u \in A(r'))$ evaluates to true; and
- satisfies $(r, r', \circ, \forall)$ if for all $u \in A(r) \cup A(r')$, the propositional formula $(u \in A(r)) \circ (u \in A(r'))$ evaluates to true.

Note that for any complete relation A and any $r \in R$, $A(r) \neq \emptyset$, so $A(r) \cup A(r') \neq \emptyset$. Thus constraints cannot be vacuously satisfied by a valid relation.

Informally speaking, universal constraints are “stronger” than existential constraints: (for any complete relation) the satisfaction of $(r, r', \circ, \forall)$ implies the satisfaction of $(r, r', \circ, \exists)$, but the converse does not hold.

We now expand these generic definitions for the four constraints defined by $(r, r', \circ, Q)$, where $\circ$ is either $\leftrightarrow$ or $\updownarrow$:

- 1) $(r, r', \leftrightarrow, \exists)$ is satisfied if there exists $u \in U$ such that $u \in A(r)$ and $u \in A(r')$; that is, $A(r) \cap A(r') \neq \emptyset$.
- 2) $(r, r', \updownarrow, \exists)$ is satisfied if there exists $u \in U$ such that either (i) $u \in A(r)$ and $u \notin A(r')$ or $u \notin A(r)$ and $u \in A(r')$; that is, $A(r) \neq A(r')$.
- 3) $(r, r', \leftrightarrow, \forall)$ is satisfied if for all $u \in A(r) \cup A(r')$, $u \in A(r)$ if and only if $u \in A(r')$; that is, $A(r) = A(r')$.
- 4) $(r, r', \updownarrow, \forall)$ is satisfied if for all $u \in A(r) \cup A(r')$, either (i) $u \in A(r)$ and $u \notin A(r')$ or (ii) $u \notin A(r)$ and $u \in A(r')$; that is, $A(r) \cap A(r') = \emptyset$.

Thus, constraints of the form $(r, r', \updownarrow, Q)$ correspond closely to the idea of separation-of-duty. Indeed, the satisfaction criterion for $(r, r', \updownarrow, \forall)$ is identical to that for a simple static separation-of-duty constraint. Similarly, constraints of the form $(r, r', \leftrightarrow, Q)$ correspond to the idea of binding-of-duty.

Now consider a constraint of the form $(r, r', \rightarrow, \forall)$. Such a constraint is satisfied if for all $u \in A(r) \cup A(r')$, $(u \in A(r)) \rightarrow (u \in A(r'))$. In other words, $A(r) \subseteq A(r')$. Thus a global constraint of this form could be used to specify a role hierarchy (in which role r' is senior to r).

Conversely, a constraint of the form $(r, r', \leftarrow, \forall)$ could be used to specify a role hierarchy in which r' is junior to r .³ Thus we can use constraints to insist that a hierarchy is strict: that is, there exists at least one user that is assigned to r' but not r . Specifically, a relation A simultaneously satisfies constraints $(r, r', \rightarrow, \forall)$ and $(r, r', \leftarrow, \forall)$ only if $A(r) \subset A(r')$.

4.2 Cardinality Constraints

We may also define *cardinality constraints*, which come in two flavors. In the following, $\triangleleft$ is a binary relation belonging to the set $\{=, <, >, \leq, \geq\}$ and t is an integer greater than 0.

- A *global* (cardinality) constraint has the form $(\triangleleft, t)$. The constraint $(\triangleleft, t)$ is satisfied by relation A if for all $r \in R$, $|A(r)| \triangleleft t$.
- A *local* (cardinality) constraint has the form $(R', \triangleleft, t)$, where $R' \subseteq R$. The constraint $(R', \triangleleft, t)$ is satisfied by relation A if $|A(R')| \triangleleft t$.

Then, for example, the global constraint $(=, 1)$ requires a valid relation A to be a function (since the number of users assigned to each resource is precisely 1), while the local constraint $(\{r\}, \leq, t)$ is a cardinality constraint in the RBAC96 sense [10] (if resource r is interpreted as a role). Finally, the k -out-of- n static separation-of-duty constraint $\text{ssod}(\{r_1, \dots, r_k\}, n)$, introduced by Li et al. [9], may be represented by the cardinality constraint $(\{r_1, \dots, r_k\}, \geq, n)$.

Remark 1. If we define a global constraint $(=, 1)$, then the universal constraint $(r, r', \circ, \forall)$ is equivalent to the existential constraint $(r, r', \circ, \exists)$ (in the sense that the former is satisfied if and only if the latter is).

4.3 User-Independent Constraints

Research on the WORKFLOW SATISFIABILITY PROBLEM has shown that the notion of *user-independent* (UI) constraints is very important. First, the class of UI constraints includes a very wide range of constraints and almost all constraints that are of relevance to access control. Thus, we believe that many constraints of interest for applications in the future will also be user-independent. Second, WSP is fixed-parameter tractable if we restrict attention to UI constraints [12]. (WSP is not FPT if we allow arbitrarily complex constraints [6].) Informally, a constraint is UI in the context of workflow satisfiability if its satisfaction only depends on the relationships that exist between users assigned to steps in a workflow (and not on the specific identities of users) [12]. We now extend the definition of user-independent used in the context of workflow satisfiability.

Let A be an authorization relation and $\sigma : U \rightarrow U$ a permutation of the user set (that is, σ is a bijection). Then, given an authorization relation $A \subseteq U \times R$, we write $\sigma(A) \subseteq U \times R$ to denote the relation $\{(\sigma(u), r) : (u, r) \in A\}$. A constraint c is *user-independent* if for every authorization relation A that satisfies c and every permutation $\sigma : U \rightarrow U$, $\sigma(A)$ satisfies c .

Consider, for example, a constraint of the form $(r, r', \leftrightarrow, \exists)$ and suppose that $A \subseteq U \times R$ satisfies the constraint. Then, by

3. Since $A(r)$ and $A(r')$ are non-empty, the constraints $(r, r', \leftarrow, \exists)$ and $(r, r', \rightarrow, \exists)$ are both equivalent to $(r, r', \leftrightarrow, \exists)$.

definition, there exists a user u such that $(u, r), (u, r') \in A$. Then, for any permutation σ , $(\sigma(u), r), (\sigma(u), r') \in \sigma(A)$, so $\sigma(A)$ also satisfies the constraint. Similar arguments may be used to show that constraints of the form $(r, r', \leftrightarrow, \forall)$, $(r, r', \uparrow, \exists)$ and $(r, r', \uparrow, \forall)$ are all UI. Equally, it is clear that global and local constraints, whose satisfaction is defined in terms of the cardinality of sets of the form $A(r)$, are UI, since a permutation (being a bijection) will preserve the cardinality of such sets. In other words, all constraints we consider in this paper are UI.

4.4 Bounded UI Constraints

We now define an important class of UI constraints that will be useful for establishing positive results in the remainder of the paper. Given a base relation A_{Bse} and a constraint c , let A be valid with respect to A_{Bse} and c . We say A *requires* v if $\{(u, r) \in A : u \neq v\}$ is not valid. (Since A is valid, this means that $\{(u, r) \in A : u \neq v\}$ is either incomplete or does not satisfy c .) Then we define

$$\text{core}(A : A_{\text{Bse}}, c) \stackrel{\text{def}}{=} \{u \in U : A \text{ requires } u\}$$

to be the *core* of A with respect to A_{Bse} and c .

Consider for instance a constraint c of the form $(r, r', \uparrow, \forall)$. If there exists an authorization relation A satisfying c , then there is one whose core contains at least two users: indeed, remove iteratively from A any pair $(u, r) \in A$ such that A does not require u . When this is no longer possible, the obtained relation has to allocate two distinct users to r and r' , both belonging to the core. The core could contain as many as k users, since the relation may allocate each resource to a different user and removing any user would compromise the completeness of the relation. However, the core cannot contain more than k users, since in any set of at least $k + 1$ users, at least two users must be allocated to the same resource and one of them could be removed without compromising the completeness or the eligibility of the relation. Conversely, for a constraint of the form $(r, r', \leftrightarrow, \forall)$ the core could contain a single user but no core contains more than $k - 1$ users, since r and r' must be assigned to the same user and any additional users may be removed without compromising completeness or eligibility.

Proposition 2. Let $\mathcal{I} = (A_{\text{Bse}}, C)$ be a satisfiable instance of D-APEP with a UI constraint $c \in C$. If A is a valid solution with respect to A_{Bse} and c then

$$|\text{core}(A : U \times R, c)| \geq |\text{core}(A : A_{\text{Bse}}, c)|.$$

Proof. We prove the more general statement that

$$\text{core}(A : A_{\text{Bse}}, c) \subseteq \text{core}(A : \Omega, c)$$

for any $\Omega \supseteq A_{\text{Bse}}$. Suppose A requires u and let $A \setminus u$ denote $\{(v, r) \in A : u \neq v\}$. Then $A \setminus u$ is either incomplete or violates c (since A is authorized, so is $A \setminus u$). If $A \setminus u$ is incomplete, then it is also incomplete for the instance (Ω, c) . The same argument holds if $A \setminus u$ violates c , which concludes the proof. $\square$

Definition 3. We say a UI constraint c is $f(k, n)$ -bounded if $|\text{core}(A : U \times R, c)| \leq f(k, n)$ for all A valid with respect to $U \times R$ and c .

TABLE 1
Upper Bounds on the Size of the Core

Constraint Type	Largest Core
$(r, r', \uparrow, \forall), (r, r', \uparrow, \exists)$	k
$(r, r', \leftrightarrow, \forall), (r, r', \rightarrow, \forall), (r, r', \leftrightarrow, \exists)$	$k - 1$
$(R', \leq, t), (\leq, t)$	k
$(R', =, t), (R', \geq, t)$	$2 \max\{k, t\}$

The definition of $f(k, n)$ -bounded constraints and Proposition 2 impose an upper bound on the number of users we need to consider when constructing candidate solutions to an instance (A_{Bse}, C) of D-APEP, and thus plays an important role in determining whether a particular variant of APEP is FPT. In particular, we show in Section 6 that D-APEP is FPT when all constraints are $f(k)$ -bounded for some function f .

We have the following elementary results, the proofs of which can be found in the appendix. The results are summarized, for ease of reference, in Table 1.

Proposition 4. Constraints $(r', r'', \rightarrow, \forall)$, $(r', r'', \leftrightarrow, \forall)$ and $(r', r'', \leftrightarrow, \exists)$ are $(k - 1)$ -bounded.

Proposition 5. Constraints $(r', r'', \uparrow, \forall)$ and $(r', r'', \uparrow, \exists)$ are k -bounded.

Proposition 6. Constraints $(R', \leq, t)$ and $(\leq, t)$ are k -bounded.

Remark 7. Obviously, Proposition 6 remains true when we replace $\leq$ with $<$. However, it does not hold if we replace $\leq$ by $=$ or $\geq$. Indeed, a constraint such as $(=, t)$ requires that some set of t users cannot be removed. Hence, if t is not bounded by a function of k only, the constraint is not $f(k)$ -bounded for any computable function f .

Proposition 8. Constraints $(R', =, t)$ and $(R', \geq, t)$ are $2 \max\{k, t\}$ -bounded, but not $(\max\{k, t\} - 1)$ -bounded.

4.5 Notation

In the remainder of this paper, we consider versions of APEP in which we restrict our attention to particular types of constraints. We use the following abbreviations for families of constraints:

- (i) **BoD** to denote the family of all existential and universal (binding-of-duty) constraints having the form $(r, r', \leftrightarrow, \exists)$ or $(r, r', \leftrightarrow, \forall)$;
- (ii) **SoD** to denote the family of all existential and universal (separation-of-duty) constraints having the form $(r, r', \uparrow, \exists)$ or $(r, r', \uparrow, \forall)$;
- (iii) **BoD_E** and **BoD_U** to denote, respectively, the family of all existential and universal binding-of-duty constraints;
- (iv) **SoD_E** and **SoD_U** to denote, respectively, the family of all existential and universal separation-of-duty constraints;
- (v) $f(k)$ -bounded to denote $f(k)$ -bounded constraints;
- (vi) G_{card} to denote the family of all global cardinality constraints;
- (vii) L_{card} to denote the family of all local cardinality constraints.

TABLE 2
Constraint Types Defined in Section 4.1

$(r, r', \leftrightarrow, \forall)$	$A(r) = A(r')$
$(r, r', \leftrightarrow, \exists)$	$A(r) \cap A(r') \neq \emptyset$
$(r, r', \uparrow, \forall)$	$A(r) \cap A(r') = \emptyset$
$(r, r', \uparrow, \exists)$	$A(r) \neq A(r')$
$(r, r', \rightarrow, \forall)$	$A(r) \subseteq A(r')$

Finally, we write, for example, $\text{APEP}(\text{BoD}, \text{L}_{\text{card}})$ to restrict the set of instances of APEP in which the set of constraints C contains only binding-of-duty and local cardinality constraints.

5 FURTHER APPLICATIONS OF APEP

In Section 6, we establish the computational complexity of a number of variants of APEP. In particular, we prove that APEP is FPT provided all constraints are $f(k)$ -bounded. In this section, we consider a number of extensions to the basic constraints we have already defined and their relevance as extensions to authorization policies. We also discuss a number of problems associated with constraints and policies from the literature and demonstrate why they may be considered to be instances of APEP, thereby establishing that these problems are FPT.

5.1 Constraint Types

In Section 4.1 we identified a number of constraint types of the form $(r, r', \circ, Q)$, where r and r' are resources, $\circ$ is a logical binary operator, and Q is a quantifier. For ease of reference we summarize these constraints and the respective conditions for satisfaction in Table 2.

We chose to introduce the constraints shown in Table 2 because of their obvious connections to known constraints in the literature and to simplify the exposition of the technical material. We now discuss ways in which these constraints could be extended. Notice that the satisfaction of each constraint may be defined in terms of $A(r)$ and $A(r')$.

One obvious extension, then, is to define constraints of the form $((r_1, \dots, r_m), \circ, Q)$, and to define constraint satisfaction in terms of $A(r_i)$, $1 \leq i \leq m$. We may define constraint satisfaction in a number of ways, including (but not limited to) the following:

- (i) for all i and j , $1 \leq i < j \leq m$, $(r_i, r_j, \circ, Q)$ is satisfied; or
- (ii) for some i and j , $1 \leq i < j \leq m$, $(r_i, r_j, \circ, Q)$ is satisfied.

Note, however, that the first of these choices can be realized simply by defining a set of constraints $\{(r_i, r_j, \circ, Q) : 1 \leq i < j \leq m\}$.

Consider the constraint $((r_1, \dots, r_m), \uparrow, \forall)$, and suppose, as another alternative for constraint satisfaction, we require that $\bigcap_{i=1}^m A(r_i) = \emptyset$. In other words, there is no user that is assigned to all resources in the set $\{r_1, \dots, r_m\}$. It is easy to see that such a constraint is k -bounded (since removing a user from a valid relation can only affect completeness, not the eligibility, of the relation). Thus, with this interpretation, $((r_1, \dots, r_m), \uparrow, \forall)$ represents a canonical SMER constraint [9] (if the set of resources is interpreted as a set of mutually exclusive roles). We return to SMER constraints in Section 5.3.

Another possible extension is to define constraints of the form $(R', R'', \circ, Q)$ and to define constraint satisfaction in terms of $A(R')$ and $A(R'')$. For example, the constraint $(R', R'', \uparrow, \forall)$ requires that $A(R') \cap A(R'') = \emptyset$. Again, constraints of this form are k -bounded. In other words, the users assigned to resources in R' are different from the users assigned to resources in R'' . This constraint, therefore, allows us to specify that resources should be allocated to disjoint teams of users (rather than just individual users). Of course such constraints could be nested: we might define a further constraint $(R'_1, R'_2, \uparrow, \forall)$ where R'_1 and R'_2 are subsets of R'' .

5.2 Resiliency in Access Control

Suppose we are given an authorization relation $A \subseteq U \times R$ and a set of resources $Q \subseteq R$. Then a resiliency policy is defined by a tuple (Q, s, d, t) , where s , d and t are integers [4]. The policy is satisfied if, following the removal of any s users from U , there exist d disjoint teams of users, $U_1, \dots, U_d$, such that $A(U_i) \supseteq Q$ and $|U_i| \leq t$ for each i .

The *resiliency checking problem* [4] asks whether a resiliency policy is satisfiable or not. It has been shown that the hard part of the problem is finding the teams (since we can enumerate all possible user sets that are missing s users), so research has focused on solving the problem for instances in which $s = 0$ [2], [4].

Informally, a solution of the resiliency checking problem may be viewed as a function mapping (different copies of the set of) resources to users. Thus we can transform an instance of the resiliency checking problem (where $s = 0$) into an instance of D-APEP. We define d copies of each resource in Q ; we write $r^{(i)}$ to denote the i th copy of resource r in Q . We then define

$$A_{\text{Bse}} = \{(u, r^{(i)}) : (u, r) \in A, 1 \leq i \leq d\}.$$

Finally, we define the global constraint $(=, 1)$ and, for all $r_1, r_2 \in Q$ and all i and j such that $1 \leq i < j \leq d$, we define a constraint $(r_1^{(i)}, r_2^{(j)}, \uparrow, \forall)$. The authorization relation A_{Bse} ensures that each user is authorized according to the original relation A . The global constraint ensures that each resource is assigned to a single user. The other constraints ensure that a user is only assigned to resources in one copy of Q .

The results in Section 6.2 assert that the resulting problem is FPT. Hence, the resiliency checking problem is also FPT (confirming an earlier result of Crampton et al. [2]).

Note, finally, that we can simplify the above construction, using the constraints introduced in Section 5.1: we use $Q^{(i)}$ to denote the i th copy of the set of resources Q and define the set of constraints

$$\{(Q^{(i)}, Q^{(j)}, \uparrow, \forall) : 1 \leq i < j \leq d\}.$$

5.3 Resiliency and Separation of Duty

The RBAC96 standard discusses constraints based on mutually exclusive roles [10]. Such a constraint is defined by a set of roles R_{mutex} and is satisfied by the user-role assignment relation provided no user is assigned to more than one role in R_{mutex} .

Li, Tripunitara and Bizri introduced the more general *static mutually exclusive role* (SMER) constraints [9], which

have the form (R_{mutex}, t) , where $t \leq |R_{\text{mutex}}|$. (A canonical SMER constraint has $t = |R_{\text{mutex}}|$.) Such a constraint is satisfied provided every user is assigned fewer than t of the roles in R_{mutex} . We can check whether a user-role assignment relation satisfies a SMER constraint in polynomial time [9], informally because we only need to consider each user once.

Li et al. went on to distinguish SMER constraints from *static separation of duty* (SSoD) policies [9], which are defined by a set of permissions P and an integer $t \leq |P|$. Such a constraint is satisfied if no subset of fewer than t users is collectively authorized for the permissions in P . Checking whether a SSoD policy is satisfied by a given user-role assignment relation is computationally hard, informally because we need to consider every possible subset of users having cardinality less than t .

Li, Wang and Tripunitara studied the complexity of determining whether it was possible to simultaneously satisfy static separation of duty constraints and a resiliency policy [4]. Unsurprisingly, it is computationally hard to decide this question, given that it is hard to decide whether an authorization relation satisfies a static separation of duty policy [9]. However, they did not consider the possibility of simultaneously satisfying SMER constraints and resiliency policies. Now observe that a SMER constraint is user-independent and is k -bounded. Thus, for example, it is possible to develop an FPT algorithm to determine whether there exists an authorization relation $A \subseteq U \times R$ such that a resiliency policy and a set of SMER constraints are simultaneously satisfied.

6 COMPLEXITY OF APEP

Before exploring the fine-grained complexity of D-APEP with respect to the different types of constraints, we first state general properties of some special cases.

First, note that adding the function constraint F to any D-APEP instance having only SoD or BoD constraints forces any solution A_{Sol} of D-APEP to be a function. In this case, D-APEP becomes equivalent to WSP. More formally, we say that a parameterized problem A is *parameter-reducible* to a parameterized problem B (and we write $A \leq_{\text{fpt}} B$) if there is a polynomial algorithm which transforms an instance (I, k) of A into an instance (I', k') of B such that (I, k) is positive for A iff (I', k') is positive for B , and $k' \leq f(k)$ for some computable function f . Clearly, if A is parameter-reducible to B and B is FPT, then A is FPT. We say that A is *parameter-equivalent* to B (and we write $A =_{\text{fpt}} B$) if $A \leq_{\text{fpt}} B$ and $B \leq_{\text{fpt}} A$. The proof of the following result is straightforward, by the definition of the function constraint F .

Theorem 9. *If D-APEP and WSP are parameterized by the number of resources and steps, respectively, we have the following:*

- $\text{D-APEP}(\text{SoD}, F) =_{\text{fpt}} \text{WSP}(=)$;
- $\text{D-APEP}(\text{BoD}, F) =_{\text{fpt}} \text{WSP}(=)$; and
- $\text{D-APEP}(\text{BoD}, \text{SoD}, F) =_{\text{fpt}} \text{WSP}(=, \neq)$.

Moreover, the following result asserts that adding BoD_U constraints to any instance of D-APEP does not change its (parameterized) complexity.

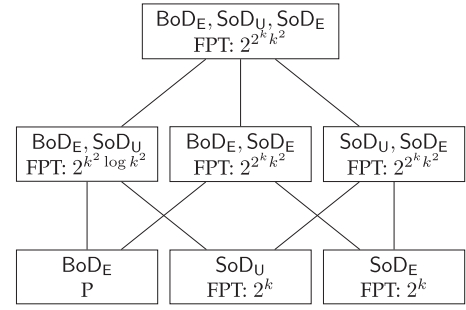


Fig. 1. Complexity of specific cases of D-APEP (polynomial factors are ignored).

Theorem 10. *Given any instance $(U, R, A_{\text{Bse}}, C)$ of $\text{D-APEP}(\text{BoD}, \text{SoD})$, one can obtain in polynomial time an instance $(U, R', A'_{\text{Bse}}, C')$ of D-APEP such that:*

- (i) C' does not contain any BoD_U constraint,
- (ii) $|A'_{\text{Bse}}| \leq |A_{\text{Bse}}|$,
- (iii) $|R'| \leq |R|$, and
- (iv) $|C'| \leq |C|$.

Proof. Let C^* be the set of BoD_U constraints from C . The idea is to consider BoD_U constraints as an equivalence relation: for $r, r' \in R$, $r \sim_b r'$ if and only if $(r, r', \leftrightarrow, \forall) \in C^*$. Now, let $R' = \{R_1, \dots, R_q\}$ be the equivalence classes of $\sim_b$. Obviously, $|R'| \leq |R|$. For all $i \in [q]$ and all $r \in R_i$, let $A'_{\text{Bse}}(r)$ denote $\bigcap_{r' \in R_i} A_{\text{Bse}}(r')$. Once again, it holds that $|A'_{\text{Bse}}| \leq |A_{\text{Bse}}|$. Finally, for every constraint $c = (r, r', \sim, Q)$ in $C \setminus C^*$ with $\sim \in \{=, \neq\}$ and $Q \in \{\forall, \exists\}$, we distinguish two cases:

- if $r \in R_i$ and $r' \in R_j$ with $i \neq j$, then add the constraint $(R_i, R_j, \sim, Q)$ (notice that in this case c is not a BoD_U constraint);
- if $r, r' \in R_i$ for some $i \in [k]$, then if c is a SoD_U or SoD_E constraint, obviously the instance is unsatisfiable (and we can output a trivially negative instance of D-APEP).

Clearly $|C'| \leq |C|$, and C' does not contain any BoD_U constraint. Finally, one can check that the output instance is satisfiable if and only if the input instance is satisfiable. $\square$

Remark 11. A corollary of Theorem 10 is that we may exclude BoD_U constraints from consideration when establishing FPT results.

In the remainder of this section, we establish that D-APEP with bounded constraints is FPT. We also propose extra algorithms for some mixed policies composed of BoD and SoD constraints in order to improve the execution time of the main algorithm for these subcases. Fig. 1 summarizes our results for D-APEP with BoD and SoD constraints.

6.1 Instances with Constraints of a Single Type

As a direct application of Theorem 10, we are able to show that $\text{D-APEP}(\text{BoD}_U)$ is polynomial-time solvable: indeed, after using the reduction of the previous result, it is clear that the instance is satisfiable if and only if A'_{Bse} is complete.

Theorem 12. *$\text{D-APEP}(\text{BoD}_U)$ is solvable in polynomial time.*

We now consider the complexity of $\text{D-APEP}(\text{SoD}_U)$ and $\text{D-APEP}(\text{BoD}_E)$.

Theorem 13. $D\text{-APEP}\langle \text{SoD}_U \rangle =_{\text{fpt}} D\text{-APEP}\langle \text{SoD}_U, F \rangle$.

Proof. It is sufficient to prove that there always exists a feasible solution of $D\text{-APEP}\langle \text{SoD}_U \rangle$ which satisfies the function constraint F .

Let us suppose $(U, R, A_{\text{Bse}}, C)$ is satisfiable, so there exists a valid solution A_{Sol} for this instance. We define another relation A' which is a function. For any $r \in R$, we have $|A_{\text{Sol}}(r)| \geq 1$, thus we can pick an arbitrary user u_r in $A_{\text{Sol}}(r)$ and set $A'(r) = \{u_r\}$. One can observe that $A' \subseteq A_{\text{Sol}} \subseteq A_{\text{Bse}}$, and thus A' is indeed a function. For a constraint $(r, r', \uparrow, \forall) \in C$, we have $A_{\text{Sol}}(r) \cap A_{\text{Sol}}(r') = \emptyset$, but since $A'(r) \subseteq A_{\text{Sol}}(r)$ and $A'(r') \subseteq A_{\text{Sol}}(r')$, we have $A'(r) \cap A'(r') = \emptyset$ as well. Thus, A' is valid for $(U, R, A_{\text{Bse}}, C')$ and is clearly a function. $\square$

Since, by Theorem 9, $D\text{-APEP}\langle \text{SoD}_U, F \rangle =_{\text{fpt}} \text{WSP}(\neq)$, and since $\text{WSP}(\neq)$ is NP-hard and FPT parameterized by the number of steps [3], we obtain the following corollary:

Corollary 14. $D\text{-APEP}\langle \text{SoD}_U \rangle$ is NP-hard and FPT parameterized by k .

In fact, it follows from [3] that $D\text{-APEP}\langle \text{SoD}_U \rangle$ can be solved in time $O^*(2^k)$. We now consider BoD_E constraints.

Theorem 15. $D\text{-APEP}\langle \text{BoD}_E \rangle$ is polynomial-time solvable.

Proof. We show that an instance $(U, R, A_{\text{Bse}}, C)$ is satisfiable iff A_{Bse} is valid. Obviously, if A_{Bse} is valid, the $D\text{-APEP}\langle \text{BoD}_E \rangle$ instance is satisfiable. Conversely observe first that A_{Bse} is obviously authorized and complete. Then, if $D\text{-APEP}\langle \text{BoD}_E \rangle$ is satisfiable, there exists $A_{\text{Sol}} \subseteq A_{\text{Bse}}$, which is valid. However, since $A_{\text{Sol}}(r) \subseteq A_{\text{Bse}}(r)$ for any $r \in R$, any constraint $(r, r', \leftrightarrow, \exists)$ satisfied by A_{Sol} is also satisfied by A_{Bse} . In other words, A_{Bse} is eligible. $\square$

6.2 Complexity of $D\text{-APEP}\langle f(k)\text{-Bounded} \rangle$

Let f be an arbitrary function in k and let $\mathcal{I} = (U, R, A_{\text{Bse}}, C)$ be a $D\text{-APEP}$ instance containing only $f(k)$ -bounded constraints. Without loss of generality, we assume that $f(k) \geq k$ (observe that all constraints considered in this paper are never better than $(k-1)$ -bounded). In this section, we introduce a method to decide whether $\mathcal{I}$ is satisfiable or not in time $O^*(2^{2^k f(k)k})$.

Given a set of resources $T \subseteq R$, we define

$$U_T = \{u \in U : A_{\text{Bse}}(u) = T\}.$$

We call U_T the (user) family associated with T . Note that for $T \neq T'$, we have $U_T \cap U_{T'} = \emptyset$. Moreover, $U = \bigcup_{T \subseteq R} U_T$. Thus $\{U_T\}_{T \subseteq R}$ is a partition of U containing at most 2^k sets. The intuition behind this definition is that when considering UI constraints (in particular, $f(k)$ -bounded constraints), all users in a set U_T play the same role. The idea of the algorithm is thus to eliminate users in “large” families to upper-bound the number of users by a function of k . To eliminate users, we apply the following reduction rule:

if there exists $T \subseteq R$ such that $|U_T| > f(k)$, then remove an arbitrary user u^ from U_T .*

	r_1	r_2	r_3
u_1, u_2	1		1
u_3, u_4, u_5, u_6, u_7	1	1	
u_8	1	1	1

Fig. 2. Use of the reduction rule.

Successive applications of this rule will result in an instance in which the number of users is at most $f(k)2^k$, a function of k only.

Consider, for example, an instance comprising the base authorization relation A_{Bse} shown in Fig. 2 ($k=3$ and $n=8$) and a single constraint $(r_1, r_2, \uparrow, \exists)$. The constraint $(r_1, r_2, \uparrow, \exists)$ is 3-bounded. There are three families of users, of which $U_{\{r_1, r_2\}}$ has cardinality greater than 3. Applying the reduction rule, we may remove users u_3 and u_4 from $U_{\{r_1, r_2\}}$.

Lemma 16. $\mathcal{I}$ is satisfiable iff $\mathcal{I}'$ is satisfiable, where $\mathcal{I}'$ is the instance obtained by applying the reduction rule to $\mathcal{I}$.

Proof. Obviously, if $\mathcal{I}'$ is satisfiable, then so is $\mathcal{I}$.

Assume $\mathcal{I}$ satisfiable and $\mathcal{I}'$ unsatisfiable, and let A be a solution for $\mathcal{I}$. Then there exists $T \subseteq R$ such that $|U_T| \geq f(k) + 1 \geq k + 1$, which means that A violates some constraint $c \in C$ (i.e. unsatisfiability of $\mathcal{I}'$ does not come from incompleteness). Since c is $f(k)$ -bounded, we have $|\text{core}(A : \mathcal{I})| \leq f(k)$ and thus $|\text{core}(A : \mathcal{I}) \cap U_T| \leq f(k)$. But, since $|U_T| \geq f(k) + 1$, and since c is user independent, we may assume, without loss of generality, that there is a user $u^* \in U_T$ such that $u^* \notin \text{core}(A : \mathcal{I})$. However, u^* is a user whose removal makes the instance unsatisfiable, a contradiction. $\square$

Theorem 17. For any computable function f depending only on k , $D\text{-APEP}\langle f(k)\text{-bounded} \rangle$ is FPT parameterized by k .

Proof. Whenever the reduction rule can be applied, we remove one user from the instance. If the rule cannot be applied, then we have an instance with at most $2^k f(k)$ users. Applying a brute force algorithm (by checking every possible relation for the reduced user set), one can check the satisfiability in time $O^*(2^{2^k f(k)k})$. $\square$

Corollary 18. $D\text{-APEP}\langle \text{SoD}, \text{BoD} \rangle$ is FPT parameterized by k .

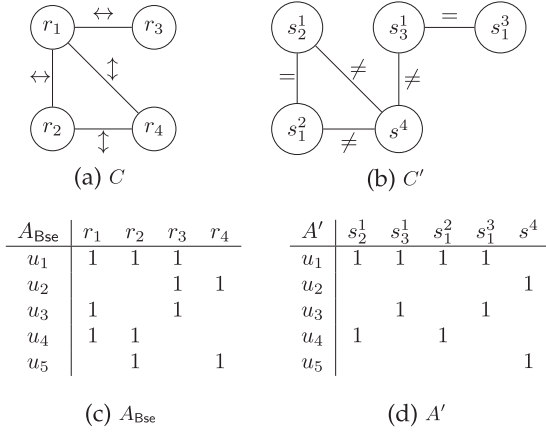
Proof. The result follows from Propositions 4, 5, and Theorem 17. $\square$

More generally, as we proved in Section 4.4 that cardinality constraints with symbol $\leq$ or $<$ are k -bounded, such constraints can be added to any $D\text{-APEP}\langle \text{SoD}, \text{BoD} \rangle$ instance without degrading the execution time. Concerning cardinality constraints with symbols $\geq$, $>$ or $=$, we have the following corollary of Proposition 8.

Corollary 19. For any computable function f depending only on k , $D\text{-APEP}\langle f(k)\text{-bounded}, \mathbf{G}_{\text{card}}, \mathbf{L}_{\text{card}} \rangle$ is FPT parameterized by k plus the maximum cardinality of all cardinality constraints.

6.3 Complexity of $D\text{-APEP}\langle \text{BoD}_E, \text{SoD}_U \rangle$

We now prove that a better running time can be obtained when considering only BoD_E and SoD_U constraints.

Fig. 3. Reducing D-APEP(BoD_E, SoD_U) to WSP(=, ≠).

Theorem 20. D-APEP(BoD_E, SoD_U) can be solved in time $\mathcal{O}(2^{k^2 \log k^2})$.

Proof. We reduce to WSP(=, ≠). Let $R = \{r_1, \dots, r_k\}$. We build a WSP(=, ≠) instance, denoted by (S', U', A', C') . We set $U' = U$. Then, for any $i \in [k]$, let

$$\Gamma(r_i) = \{j \in [k] : (r_i, r_j, \leftrightarrow, \exists) \in C\}.$$

For each resource $r_i \in R$, we introduce a set of steps S^i . If $\Gamma(r_i) = \emptyset$, then $S^i = \{s^i\}$. Otherwise, $S^i = \{s_j^i : j \in \Gamma(r_i)\}$. Then define $S' = \bigcup_{i \in [k]} S^i$. Observe that $|S'| \leq k(k-1)$. We then define the following constraints:

- For any $i \in [k]$, if $\Gamma(r_i) \neq \emptyset$, then for all $j \in \Gamma(r_i)$, we add the constraint $(s_j^i, s_i^j, =)$.
- For any $i, j \in [k]$, if $(r_i, r_j, \uparrow, \forall) \in C$, then, for every $s \in S^i$ and every $s' \in S^j$, we add the constraint $(s, s', \neq)$.

Finally, we define the authorization policy A' . For every $i \in [k]$ and every $u \in U$:

- If $\Gamma(r_i) = \emptyset$, then $(u, s^i) \in A'$ iff $(u, r_i) \in A_{Bse}$.
- If $\Gamma(r_i) \neq \emptyset$, then $\forall j \in \Gamma(r_i)$, $(u, s_j^i) \in A'$ iff $(u, r_i), (u, r_j) \in A_{Bse}$.

The construction is illustrated in Fig. 3 for a small example. Clearly this construction can be carried out in polynomial time.

Let us suppose there is a valid plan $\pi : S' \rightarrow U'$ for (S', U', A', C') . We set $A_\pi = \{(u, r_i) : i \in [k], \pi(s) = u \text{ for some } s \in S^i\}$. One can observe that A_π is authorized and complete. Then, for any $i, j \in [k]$ such that $(r_i, r_j, \leftrightarrow, \exists) \in C$, we must have $\pi(s_j^i) = \pi(s_i^j) = u$ for some $u \in U$, which implies that $u \in A_\pi(r_i) \cap A_\pi(r_j)$. Then, if $(r_i, r_j, \uparrow, \forall) \in C$, we know that $\pi(s) \neq \pi(s')$ for any $s \in S^i$ and any $s' \in S^j$, which implies that $A_\pi(r_i) \cap A_\pi(r_j) = \emptyset$. This proves that A_π is valid.

Conversely, we suppose (U, R, A_{Bse}, C) is satisfiable, and let A_{Sol} be a valid solution. For any $i \in [k]$, we have the following:

- If $\Gamma(r_i) = \emptyset$, then define $\pi(s^i)$ to be an arbitrary user in $A_{Sol}(r_i)$.
- If $\Gamma(r_i) \neq \emptyset$, then, for every $j \in \Gamma(r_i)$, define $\pi(s_j^i)$ as an arbitrary user in $A_{Sol}(r_i) \cap A_{Sol}(r_j)$, and set also $\pi(s_j^i) = \pi(s_i^j)$.

One can observe that π is authorized and complete. By construction, every constraint $(s_j^i, s_i^j, =) \in C'$ is satisfied. Finally, for every $s, s' \in S'$ such that $(s, s', \neq)$, it must be the case that $s \in S^i$ and $s' \in S^j$ such that $(r_i, r_j, \uparrow, \forall) \in C$, which implies that $A_{Sol}(r_i) \cap A_{Sol}(r_j) = \emptyset$. Hence we must have $\pi(s) \neq \pi(s')$, and π is a valid plan. $\square$

6.4 Complexity of MAXAPEP(SoD_U)

We now introduce a particular version of O-APEP, which seeks to find a valid authorization relation of maximum cardinality. We write M_{Sol} to denote the cardinality of such a relation. Such a relation is, in some sense, a most resilient authorization relation possible, given the authorization constraints. We call this problem MAXAPEP. (We may also define a decision version APEP to find resilient authorization relations. We may, for example, introduce a global constraint $(\geq, t)$, which requires that at least t users are authorized for each resource. These types of problems are related to notions of resiliency in workflow systems [6].)

In this section, (A_{Bse}, C) is an APEP(SoD_U) instance. In Theorem 13, we established that D-APEP(SoD_U) could be reduced to D-APEP(SoD_U, F). Let Π denote the set of valid solutions to instance $(A_{Bse}, C \cup \{(=, 1)\})$ (that is, functions $\pi : R \rightarrow U$). Given a function $\pi \in \Pi$, we say $A \subseteq U \times R$ contains π if and only if for every $r \in R$, $(\pi(r), r) \in A$. Let M_π denote the maximum size of a valid authorization relations containing π . Theorem 13 established that any solution A_{Sol} of APEP(SoD_U) contains at least one function $\pi \in \Pi$. We write M_{Sol} to denote $\max\{M_\pi : \pi \in \Pi\}$.

6.4.1 Patterns

A function $\pi : R \rightarrow U$ defines an equivalence relation $\sim_\pi$ on R , where $r \sim_\pi r'$ iff $\pi(r) = \pi(r')$. The equivalence classes defined by this relation form a partition of R which we call the *pattern* associated with π and denote it by $P(\pi)$. We say two functions π and π' are *equivalent* if $P(\pi) = P(\pi')$. For UI constraints and any two functions π and π' such that $P(\pi) = P(\pi')$, π is eligible iff π' is eligible. Hence, we will say P is *eligible* if and only if, there exists π such that $P = P(\pi)$ and π is eligible for C . Henceforth, we only consider eligible patterns. We write M_P to denote $\max\{M_\pi : P(\pi) = P\}$. There exists an eligible pattern P such that $M_{Sol} = M_P$.

Let us suppose that we are able, given a pattern P , to construct a valid A , such that $|A| = M_P$, in FPT time $f(k)n^{O(1)}$. There are at most B_k eligible patterns, where B_k is the Bell number and $B_k = O(2^{k \log k})$ [23].⁴ Then, MAXAPEP(SoD_U) would be FPT: exploring all the eligible patterns and applying the FPT algorithm to compute M_P for each P is executed in time $O^*(2^{k \log k} f(k))$. As a consequence, our objective now is to design a FPT algorithm to compute A_P such that $|A_P| = M_P$.

6.4.2 Exploring Patterns to Solve MAXAPEP(SoD_U)

Lemma 21. Let $P = \{T_1, T_2, \dots, T_d\}$ be a pattern. An authorization relation A_P , such that $|A_P| = M_P$, can be computed in FPT time $O^*(2^k)$.

Proof. Clearly $d \leq n$. We extend P into P^* in order to have $|P^*| = n$. We set $P^* = \{T_1, \dots, T_d, \emptyset_1, \dots, \emptyset_{n-d}\} = \{T_1, \dots, T_n\}$.

4. All logarithms in this paper are of base 2.

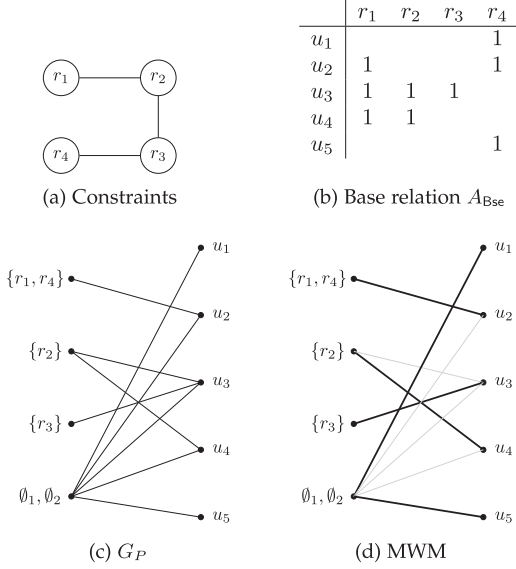


Fig. 4. Computing a maximum weighted matching for an instance of MAXAPEP(SoDU).

We build a weighted bipartite graph $G_P = (P^* \cup U, E, \omega)$, where $(T_i, u) \in E$ if and only if $T_i \subseteq A_{Bse}(u)$ and for any $i \in [n - d]$ and $u \in U$, $(\emptyset_i, u) \in E$. Assign to $(T_i, u) \in E$ weight $\omega(T_i, u)$ which is the cardinality of the maximum independent set in $A_{Bse}(u)$ containing T_i :

$$\omega(T_i, u) = \max_{\substack{\forall (r, r') \in X^2, (r, r', \uparrow, \exists) \notin C \\ T_i \subseteq X \subseteq A_{Bse}(u)}} |X|.$$

There are at most n^2 weights to compute. For any $e \in E$, calculation of every $\omega(e)$ can be performed in time $O(2^k)$ by enumerating all subsets of $A_{Bse}(u)$. Thus the bipartite graph G_P can be built in time $O^*(2^k)$. We solve the ASSIGNMENT PROBLEM on G_P and obtain a maximum weighted matching (MWM), $\mathcal{M}$, in polynomial time using the Hungarian algorithm [24].

For every edge $e \in \mathcal{M}$ we compute an independent set X_e as follows. For each $(T_i, u) \in \mathcal{M}$, choose a maximum independent set $X_{(T_i, u)}$ such that $T_i \subseteq X_{(T_i, u)} \subseteq A_{Bse}(u)$ and, therefore, $|X_{(T_i, u)}| = \omega(T_i, u)$. We define the authorization relation $A_{\mathcal{M}}$ such that $A_{\mathcal{M}}(u) = X_{(T_i, u)}$.

For any $u \in U$, $A_{\mathcal{M}}(u) \subseteq A_{Bse}(u)$. Furthermore, for any $u \in U$, $A_{\mathcal{M}}(u)$ contains resources which are pairwise independent, so $A_{\mathcal{M}}$ is valid. We define the function $\tilde{\pi}$ such that $\tilde{\pi}(r) = u$ if and only if $r \in T_i$ and $(T_i, u) \in \mathcal{M}$. $A_{\mathcal{M}}$ contains $\tilde{\pi}$ whose pattern is P . Thus, $|A_{\mathcal{M}}| \leq M_P$.

We know that there exists a valid function π such that $M_P = M_\pi$ and $P(\pi) = P = P(\tilde{\pi})$. There exists a matching $\mathcal{M}'$ representing π in G_P . If $\pi(T_i) = u$, then $(T_i, u) \in \mathcal{M}'$. If $\pi^{-1}(u)$ is empty, we associate u with an arbitrary vertex $\emptyset_i$. Since $|A_{\mathcal{M}}|$ is equal to the weight of the MWM $\mathcal{M}$ of G_P , we have $M_\pi \leq |A_{\mathcal{M}}|$. Hence, $M_P = |A_{\mathcal{M}}|$. $\square$

In Fig. 4, we use a simple example to illustrate the matching process described in the proof of Lemma 21. We consider the pattern $P = \{\{r_1, r_4\}, \{r_2\}, \{r_3\}\}$ and merge $\emptyset_1$ and $\emptyset_2$ into a single node to keep the bipartite graph readable. The figure shows G_P (derived from A_{Bse}) and the resulting MWM (where the matching is indicated by the thick lines). Then, for

example, $\omega(\{r_3\}, u_3) = 2$ because $X = \{r_1, r_3\}$ is the largest independent subset of $A_{Bse}(u_3)$ containing r_3 .

Theorem 22. MAXAPEP(SoDU) (and thus D-APEP(SoDU)) can be solved in FPT time $O^*(2^{k+k \log k})$.

Proof. We explore all eligible patterns P . For each one, we construct A_P using Lemma 21, and keep the largest one. The time complexity of this algorithm is $O^*(2^{k+k \log k})$. Hence, it is FPT parameterized by k . $\square$

6.5 Complexity of D-APEP(SoDE)

In this section, we solve the MAXAPEP(SoDE) problem by reducing to the MAX WEIGHTED PARTITION problem [25]: that is, given a ground set K and p functions $f_1, \dots, f_p$ from 2^K to integers from the range $[-M, M]$, $M \geq 1$, find a partition $\{K_1, \dots, K_p\}$ of K that maximizes $\sum_{i=1}^p f_i(K_i)$. The following result is a corollary of the main theorem on MAX WEIGHTED PARTITION in [25].

Lemma 23. MAX WEIGHTED PARTITION can be solved in time $O^*(2^k p^2 M)$.

Theorem 24. D-APEP(SoDE) and MAXAPEP(SoDE) can be solved in time $O^*(2^k)$.

Proof. We reduce to the MAX WEIGHTED PARTITION problem. The ground set is R , and we construct weight functions indexed by sets, which are elements of a family χ of subsets of U . We say that two resources $r, r' \in R$ are independent if and only if $(r, r', \uparrow, \exists) \notin C$. Moreover, the degree $d(r)$ of a resource $r \in R$ is the number of resources r' such that $(r, r', \uparrow, \exists) \in C$. The index set χ is defined as $\bigcup_{r \in R} \chi_r$, where χ_r consists of all subsets of $A_{Bse}(r)$ if $|A_{Bse}(r)| \leq \log k$, or $d(r) + 1$ largest subsets of $A_{Bse}(r)$ if $|A_{Bse}(r)| > \log k$ (breaking ties arbitrarily). For every $X \in \chi$, we define a weight function $f_X : 2^R \rightarrow [-|A_{Bse}| - 1, |A_{Bse}|]$ as follows: for every $T \subseteq R$, $f_X(T)$ is set to $|T||X|$ if T is an independent set and $X \subseteq A_{Bse}(r)$ for every $r \in T$, and $f_X(T) = -|A_{Bse}| - 1$ otherwise. Now, we show that any valid authorization relation A_{Sol} of D-APEP(SoDE) corresponds to a partition of R of cost $|A_{Sol}|$, and vice versa.

Let $\mathcal{T} = \{T_X : X \in \chi\}$ be a partition of R of nonnegative weight. Note that if $f_X(T_X) \geq 0$ for every $X \in \chi$ then $\sum_{X \in \chi} f_X(T_X) \leq |A_{Bse}|$. Thus, since $\mathcal{T}$ is of nonnegative weight, $f_X(T_X) \geq 0$ for every $X \in \chi$. Construct an authorization relation A_{Sol} such that for any $X \in \chi$, for any $r \in T_X$, we have $A_{Sol}(r) = X$. Obviously, since $\mathcal{T}$ is a partition of R , A_{Sol} is complete. Then, by definition of χ , we have $A_{Sol}(r) \subseteq A_{Bse}(r)$ and thus A_{Sol} is authorized. Finally, for any $r, r' \in R$ such that $A_{Sol}(r) = A_{Sol}(r')$, it must hold that $r, r' \in T_{A_{Sol}(r)}$, and since $T_{A_{Sol}(r)}$ is independent, $(r, r', \uparrow, \exists) \notin C$, and A_{Sol} is eligible. In other words, A_{Sol} is a valid authorization relation, and its weight is $\sum_{X \in \chi} f_X(T_X)$.

For any valid authorization relation A_{Sol} , let $P(A_{Sol})$ be the partition of R into equivalence classes with respect to the following equivalence relation: $r, r' \in R$ are equivalent if and only if $A_{Sol}(r) = A_{Sol}(r')$. We now prove that there always exists a valid authorization relation A'_{Sol} of size at least $|A_{Sol}|$ such that $A'_{Sol}(r) \in \chi$ for every $r \in R$. If this is true, then it will mean that we may assume that

$P(A_{\text{Sol}}) = \{T_X : X \in \chi\}$, and since A_{Sol} is valid, $\sum_{X \in \chi} f_X(T_X) = |A_{\text{Sol}}|$. For $r \in R$, if $|A_{\text{Bse}}(r)| \leq \log k$, then since χ contains all subsets of $A_{\text{Bse}}(r)$, it holds that $A_{\text{Sol}}(r) \in \chi$. If $|A_{\text{Bse}}(r)| > \log k$ and $A_{\text{Sol}}(r) \notin \chi_r$, recall that χ_r consists of $d(r) + 1$ largest subsets of $A_{\text{Bse}}(r)$. Hence, there must exist $X \in \chi_r$ such that $A_{\text{Sol}}(r') \neq X$ for every r' such that $(r, r', \uparrow, \exists) \in C$. Hence, replacing $A_{\text{Sol}}(r)$ by X creates another valid authorization relation A'_{Sol} of size at least $|A_{\text{Sol}}|$ and such that $A'_{\text{Sol}}(r) \in \chi_r$. Repeating this modification for every $r \in R$ such that $A_{\text{Sol}}(r) \notin \chi_r$, we end up with a valid authorization having the desired property.

Using the reduction above together with Lemma 23, we prove the claimed statement. $\square$

7 CONCLUSION

In this paper we have introduced a general framework within which we can specify problems concerned with finding authorization relations (“policies”) that must satisfy certain kinds of constraints. We have shown that there exist FPT algorithms to solve the authorization policy existence problem when all constraints are user-independent and are bounded in an appropriate way. We have also shown that many constraints of practical interest are indeed user-independent and bounded.

Our prior work on implementing FPT algorithms for the workflow satisfiability problem [26] suggests that the theoretical results obtained in this paper should translate to relatively efficient algorithms in practice; certainly far more efficient than a brute-force approach to solving APEP. We hope to confirm this expectation by developing practical implementations of the algorithms described in this paper.

Although we have chosen to consider user-independent constraints, not least because such constraints have been studied extensively in the literature on workflow satisfiability, we could equally well consider resource-independent constraints, because our framework is symmetric in a way that workflow satisfiability questions are not. So, for example, we could define a constraint of the form $(u, u', \uparrow, \forall)$ which would be satisfied provided the set of resources assigned to u is distinct from the set of resources assigned to u' . In this way, we search for authorization relations that guarantee certain users do not have access to the same resources. Moreover, if the number of users is small relative to the number of resources, which may well be the case in some multi-user systems (such as file systems), then n will be the small parameter and the symmetry of our framework admits FPT algorithms for solving problem instances of this form.

We believe there are many opportunities for future work, not least exploring what types of authorization constraints might be useful in practice and determining whether those constraints are user-independent and bounded.

APPENDIX

Proof of Proposition 4. Let c be one of $(r', r'', \rightarrow, \forall)$, $(r', r'', \leftrightarrow, \forall)$ or $(r', r'', \leftrightarrow, \exists)$. Let V be a set of $k - 1$ distinct users and consider A , where $A(r') = A(r'') = \{u\}$ for

some $u \in V$ and, for any $r_1, r_2 \in R \setminus \{r', r''\}$, $|A(r_1)| = 1$, $A(r_1) \neq A(r')$, and $A(r_1) \neq A(r_2)$. Then $(A : U \times R, c) = V$ and $|(A : U \times R, c)| = k - 1$. Moreover, for any relation A' valid with respect to $U \times R$ and c , any subset of $A'(R)$ of size at least k must contain two users who are both assigned to the same resource; thus one of them can be removed without affecting completeness or satisfiability. Hence, the constraint is $(k - 1)$ -bounded. $\square$

Proof of Proposition 5. Let V be a set of k distinct users and consider A where $|A(r)| = 1$ for each $r \in R$ and $A(R) = V$. Then $(A : U \times R, c) = V$ and $|(A : U \times R, c)| = k$. Now, for any relation A' valid with respect to $U \times R$ and c , any subset of $A'(R)$ of size at least $k + 1$ must contain two users who are both assigned to the same resource, and thus one of them can be removed without violating completeness or satisfiability. Thus $|(A' : U \times R, c)| \leq |(A : U \times R, c)|$, from which the result follows. $\square$

Proof of Proposition 6. Given any valid solution A , the removal of any user cannot make A non-eligible with respect to $c = (R', \leq, t)$, but may violate completeness. Hence, $(A : U \times R, c)$ is largest when $|A(r)| = 1$ for all r and $A(R) = k$, in which case we have $|(A : U \times R, c)| = k$. (The global cardinality constraint $c = (\leq, t)$ is k -bounded because, in this case too, no removal affects the eligibility of the relation; it can only affect the completeness.) $\square$

Proof of Proposition 8. We only give the proof for $(R', =, t)$, the other one being similar. One can observe that $(A : U \times R, c)$ is largest when $|A(r)| = 1$ for any $r \in R \setminus R'$, $|A(R \setminus R')| = |R \setminus R'|$, and $A(R') \cap A(R \setminus R') = \emptyset$. In this case we have $|(A : U \times R, c)| \leq |R \setminus R'| + t \leq 2 \max\{k, t\}$.

Concerning the negative result, observe that if $\max\{k, t\} = t$, then no user of $A(R')$ can be removed from any valid solution, and if $\max\{k, t\} = k$, then there exists solutions in which $A(R) \geq k$ and the removal of any user from $A(R)$ either violates a constraint or breaks completeness. $\square$

REFERENCES

- [1] P. Bergé, J. Crampton, G. Z. Gutin, and R. Watrigant, “The authorization policy existence problem,” in *Proc. 7th ACM Conf. Data Appl. Secur. Privacy*, 2017, pp. 163–165. [Online]. Available: <http://doi.acm.org/10.1145/3029806.3029844>
- [2] J. Crampton, G. Gutin, and R. Watrigant, “Resiliency policies in access control revisited,” in *Proc. 21st ACM Symp. Access Control Models Technol.*, 2016, pp. 101–111. [Online]. Available: <http://doi.acm.org/10.1145/2914642.2914650>
- [3] J. Crampton, G. Gutin, and A. Yeo, “On the parameterized complexity and kernelization of the workflow satisfiability problem,” *ACM Trans. Inf. Syst. Secur.*, vol. 16, no. 1, 2013, Art. no. 4. [Online]. Available: <http://doi.acm.org/10.1145/2487222.2487226>
- [4] N. Li, Q. Wang, and M. V. Tripunitara, “Resiliency policies in access control,” *ACM Trans. Inf. Syst. Secur.*, vol. 12, no. 4, 2009.
- [5] J. C. Mace, C. Morisset, and A. P. A. van Moorsel, “Quantitative workflow resiliency,” in *Proc. 19th Eur. Symp. Res. Comput. Secur.*, 2014, pp. 344–361. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-11203-9_20
- [6] Q. Wang and N. Li, “Satisfiability and resiliency in workflow authorization systems,” *ACM Trans. Inf. Syst. Secur.*, vol. 13, no. 4, 2010, Art. no. 40.
- [7] D. F. C. Brewer and M. J. Nash, “The Chinese wall security policy,” in *Proc. IEEE Symp. Secur. Privacy*, 1989, pp. 206–214.

- [8] V. D. Gligor, S. I. Gavrila, and D. F. Ferraiolo, "On the formal definition of separation-of-duty policies and their composition," in *Proc. IEEE Symp. Secur. Privacy*, 1998, pp. 172–183. [Online]. Available: <http://doi.ieeecomputersociety.org/10.1109/SECPRI.1998.674833>
- [9] N. Li, M. V. Tripunitara, and Z. Bizri, "On mutually exclusive roles and separation-of-duty," *ACM Trans. Inf. Syst. Secur.*, vol. 10, no. 2, 2007, Art. no. 5.
- [10] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Comput.*, vol. 29, no. 2, pp. 38–47, Feb. 1996. [Online]. Available: <http://dx.doi.org/10.1109/2.485845>
- [11] D. A. Basin, S. J. Burri, and G. Karjoth, "Obstruction-free authorization enforcement: Aligning security and business objectives," *J. Comput. Secur.*, vol. 22, no. 5, pp. 661–698, 2014. [Online]. Available: <http://dx.doi.org/10.3233/JCS-140500>
- [12] D. Cohen, J. Crampton, A. Gagarin, G. Gutin, and M. Jones, "Iterative plan construction for the workflow satisfiability problem," *J. Artif. Intell. Res.*, vol. 51, pp. 555–577, 2014. [Online]. Available: <http://dx.doi.org/10.1613/jair.4435>
- [13] R. G. Downey and M. R. Fellows, *Fundamentals of Parameterized Complexity*. New York, NY, USA: Springer, 2013.
- [14] R. Niedermeier, *Invitation to Fixed-Parameter Algorithms*. London, U.K.: Oxford Univ. Press, 2006.
- [15] J. Crampton, G. Gutin, and D. Karapetyan, "Valued workflow satisfiability problem," in *Proc. 20th ACM Symp. Access Control Models Technol.*, 2015, pp. 3–13.
- [16] D. Karapetyan, A. Gagarin, and G. Gutin, "Pattern backtracking algorithm for the workflow satisfiability problem with user-independent constraints," in *Proc. 9th Int. Workshop Frontiers Algorithmics*, 2015, pp. 138–149.
- [17] E. Bertino, E. Ferrari, and V. Atluri, "The specification and enforcement of authorization constraints in workflow management systems," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 1, pp. 65–104, 1999.
- [18] J. Crampton, "A reference monitor for workflow systems with constrained task execution," in *Proc. 10th ACM Symp. Access Control Models Technol.*, 2005, pp. 38–47.
- [19] A. Schaad, V. Lotz, and K. Sohr, "A model-checking approach to analysing organisational controls in a loan origination process," in *Proc. 11th ACM Symp. Access Control Models Technol.*, 2006, pp. 139–149. [Online]. Available: <http://doi.acm.org/10.1145/1133058.1133079>
- [20] J. Crampton, A. V. Gagarin, G. Gutin, and M. Jones, "On the workflow satisfiability problem with class-independent constraints," in *Proc. 10th Int. Symp. Parameterized Exact Comput.*, 2015, pp. 66–77. [Online]. Available: <http://dx.doi.org/10.4230/LIPIcs.IPEC.2015.66>
- [21] R. T. Simon and M. E. Zurko, "Separation of duty in role-based environments," in *Proc. 10th Comput. Secur. Found. Workshop*, 1997, pp. 183–194. [Online]. Available: <http://dx.doi.org/10.1109/CSFW.1997.596811>
- [22] A. Roy, S. Sural, A. K. Majumdar, J. Vaidya, and V. Atluri, "Minimizing organizational user requirement while meeting security constraints," *ACM Trans. Manage. Inf. Syst.*, vol. 6, no. 3, 2015, Art. no. 12. [Online]. Available: <http://doi.acm.org/10.1145/2811269>
- [23] D. Berend and T. Tassa, "Improved bounds on Bell numbers and on moments of sums of random variables," *Probability Math. Statist.*, vol. 30, no. 2, pp. 185–205, 2010.
- [24] H. W. Kuhn, "The Hungarian method for the assignment problem," in *Proc. 50 Years Integer Program. 1958–2008 - From Early Years State-of-the-Art*, 2010, pp. 29–47.
- [25] A. Björklund, T. Husfeldt, and M. Koivisto, "Set partitioning via inclusion-exclusion," *SIAM J. Comput.*, vol. 39, no. 2, pp. 546–563, 2009.
- [26] D. A. Cohen, J. Crampton, A. Gagarin, G. Z. Gutin, and M. Jones, "Algorithms for the workflow satisfiability problem engineered for counting constraints," *J. Comb. Optim.*, vol. 32, no. 1, pp. 3–24, 2016. [Online]. Available: <https://doi.org/10.1007/s10878-015-9877-7>

Pierre Bergé received the graduate degree from CentraleSupélec, France. He is working toward the PhD degree in computer science in LRI, Université Paris-Sud, Université Paris-Saclay, France. His research interests include combinatorial optimization, graph theory and complexity.

Jason Crampton is a professor in information security with Royal Holloway, University of London, United Kingdom. His research focuses on access control, particularly languages for specifying access control policies and cryptographic enforcement of policies.

Gregory Gutin received the PhD degree in mathematics from Tel Aviv University, Israel, in 1993. Since 2000 he has been a professor of computer science with Royal Holloway, University of London, United Kingdom. His research interests include access control, combinatorial optimization, graph theory and applications, and parameterized algorithms and complexity.

Rémi Watrigant is an assistant professor with Université Claude Bernard Lyon 1, doing research in the LIP laboratory of ENS Lyon. His research interest includes algorithmics, mainly for graph optimization problems.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**